



Task 4 — Design Note

1. Production Failure Modes

Failure Mode 1 — Knowledge Base Retrieval Failure

The first major failure mode is incorrect or incomplete knowledge-base retrieval. If the vector database is empty, documents are missing, embeddings fail, or the retrieval query returns irrelevant chunks, the LLM may classify a ticket without sufficient grounding. This can result in incorrect `known_issue` values or irrelevant knowledge-base references.

I would detect this using retrieval metrics such as document count, similarity/relevance thresholds, retrieval latency, and the percentage of requests receiving zero relevant documents. Application logs and monitoring alerts would identify sudden drops in retrieval quality.

Mitigation would include validating the knowledge-base directory during startup, automatically indexing new documents, validating embeddings, setting a minimum relevance threshold, and returning an explicit "insufficient knowledge-base evidence" state instead of pretending that an issue is known.

Failure Mode 2 — LLM Output or API Failure

Gemini can experience API timeouts, rate limits, unavailable models, malformed responses, or temporary service failures. Since the final classification depends on the model, this can directly affect ticket processing.

The system should monitor API latency, error rates, timeout rates, token usage, and invalid structured responses. Requests should use timeouts and controlled retries with exponential backoff.

For mitigation, the application should validate the model response using Pydantic schemas. Invalid responses should not be returned as successful classifications. A fallback model or deterministic rule-based classifier can be used for critical fields such as P1 detection and known error codes.

Failure Mode 3 — Incorrect Classification / Hallucination

Even with retrieved documents, an LLM can misunderstand a ticket or invent technical details. This is especially important for adversarial tickets where the customer describes dissatisfaction without an actual outage.

Detection should include evaluation datasets, confidence/relevance thresholds, human review for high-impact P1 classifications, and monitoring of disagreement between retrieval evidence and model output.

Mitigation includes strict prompts, structured JSON output, grounding responses in retrieved context, explicit instructions not to invent facts, and deterministic rules for high-confidence signals such as documented error codes and critical outage indicators.

2. Latency vs Quality

One concrete trade-off in this solution is the number of knowledge-base chunks retrieved before sending context to Gemini.

Retrieving more chunks can improve recall because the model receives more potentially relevant information. However, larger context increases embedding/search processing, prompt size, model processing time, and API cost.

The current design uses a small top-K retrieval strategy rather than sending the complete knowledge base to Gemini. This provides a practical balance between response quality and latency.

If latency became the hard constraint, I would reduce top-K, cache embeddings and frequent queries, pre-compute document metadata, use a faster embedding model, and introduce deterministic rules for common issues. For example, an error such as `SESSION_INVALID` can be matched directly against a structured error-code index before invoking the LLM.

This hybrid approach would allow obvious tickets to be processed quickly while preserving RAG + LLM reasoning for ambiguous tickets.

3. Data Sensitivity and PII Protection

Customer tickets and account records may contain personally identifiable information such as names, email addresses, company information, account identifiers, and potentially confidential business details.

The design follows a data-minimization principle. Only the information required for classification and retrieval should be sent to an external LLM API. Sensitive fields that are not necessary for reasoning should be removed or masked before the API request.

For production, I would add a PII detection/redaction layer before the Gemini call. Examples include masking email addresses, phone numbers, personal names, credentials, API keys, and authentication tokens.

API keys and secrets are stored in environment variables rather than source code or GitHub. `.env` is excluded through `.gitignore`.

The vector database should also be protected with appropriate access controls and encryption. Production logs must avoid storing complete ticket bodies or secrets. Instead, logs should contain request IDs, latency, status, retrieval counts, and other non-sensitive operational metadata.

Where organizational requirements prohibit external processing of customer data, the architecture can be adapted to use an enterprise-approved or self-hosted model while keeping the RAG and application layers unchanged.

4. Scaling to 10× Ticket Volume

At 10× the current ticket volume, the first bottlenecks would likely be LLM API throughput, embedding generation, request concurrency, and synchronous API processing.

A single FastAPI instance could become CPU- and I/O-constrained as concurrent requests increase. Gemini API rate limits could also become a bottleneck, while repeatedly generating embeddings for the same documents would waste resources.

The first scaling improvement would be to separate ingestion from online ticket processing.

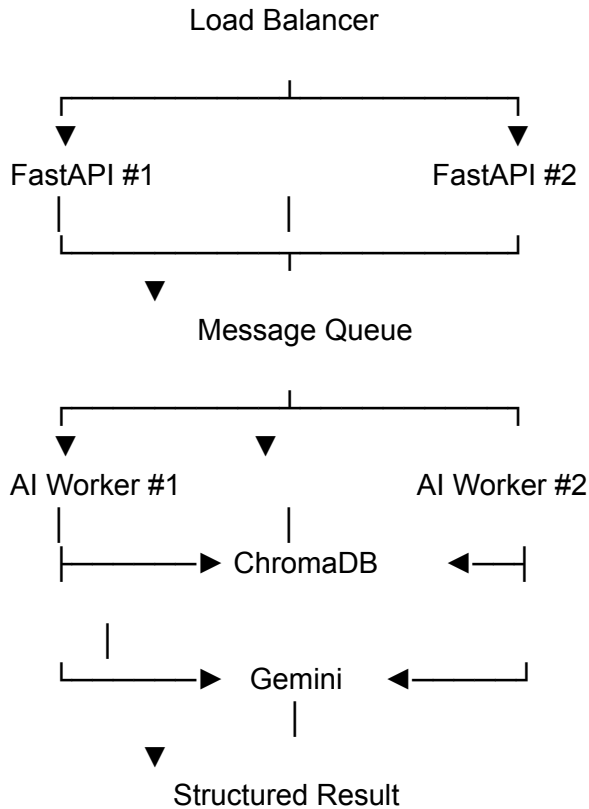
Knowledge-base indexing should run as an asynchronous background job rather than during user requests. Document embeddings should be generated once and reused.

The API layer can then be horizontally scaled using multiple FastAPI workers or containers behind a load balancer.

For higher volumes, a queue such as Redis/Celery or a cloud message queue can decouple ticket ingestion from AI processing. Worker processes can consume tickets asynchronously and call the LLM with controlled concurrency.

Caching can also reduce repeated work for identical or highly similar tickets. Rate limiting and exponential backoff would protect external APIs.

The scalable architecture would therefore become:



This architecture allows the number of API servers and AI workers to scale independently as ticket volume increases.



Security Considerations

- API keys are stored in environment variables.
 - `.env` should never be committed to GitHub.
 - Customer PII should be minimized before external API calls.
 - Authentication tokens and credentials must never be included in logs.
 - Production ChromaDB access should be protected.
 - API requests should use HTTPS in production.
 - Input and output schemas should be validated.
 - High-impact classifications such as P1 should support human review.
-



Future Improvements

Potential improvements include:

- Hybrid keyword + vector retrieval
 - Structured error-code matching
 - Cross-encoder reranking
 - PII detection and automatic redaction
 - Human-in-the-loop approval for P1 tickets
 - Redis caching
 - Async ticket processing
 - Queue-based architecture
 - Automated knowledge-base ingestion
 - Retrieval evaluation metrics
 - LLM response confidence scoring
 - Observability with metrics and tracing
 - Production authentication and authorization
-

Conclusion

This project demonstrates an end-to-end AI support intelligence workflow combining **LLM reasoning, Retrieval-Augmented Generation, vector search, structured outputs, and account-level analytics.**

The evaluation achieved:

11 / 12 tests passed

92% overall pass rate

0.93 overall quality score

The architecture is intentionally modular so that the RAG, classification, account analysis, UI, and API layers can be independently improved and scaled for production use.